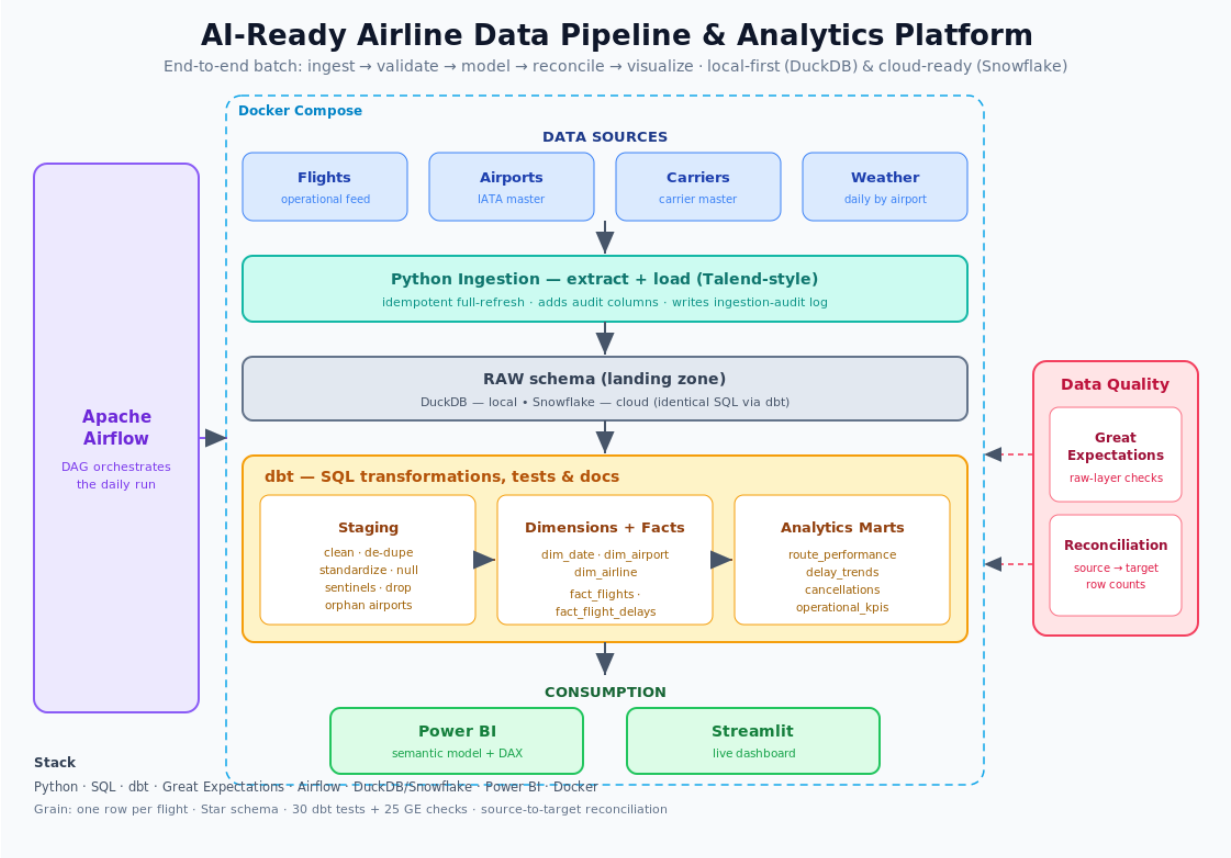
AI-Ready Airline Data Pipeline & Analytics Platform

Data Engineering Portfolio — Project Report

End-to-end platform that ingests airline operational data, validates and models it into a tested star schema, and serves analytics — running locally on DuckDB and shipping to Snowflake unchanged.



Python · SQL · dbt · Apache Airflow · Great Expectations · DuckDB / Snowflake · Power BI · Docker

Author: Srikanth Banoth | Mobile: 9392327533 | Email: banothsrikanth75338@gmail.com

1. Executive summary

This project is a production-shaped data platform for airline operations. It ingests four sources — flights, airports, carriers and weather — cleans and validates them, models them into a Kimball star schema with dbt, and exposes the results through a Power BI semantic model and a runnable Streamlit dashboard. The entire pipeline is orchestrated by Apache Airflow and packaged with Docker. Its defining property is **local-first, cloud-ready**: it runs end-to-end on a laptop against an embedded DuckDB warehouse, yet the identical dbt models target Snowflake by switching a single profile.

The goal was not just to move data, but to do it the way a real data team does: layered modelling, enforced data quality, referential-integrity testing, row-count reconciliation, orchestration, and BI — each an explicit, inspectable part of the repository.

2. What I built

- **Ingestion (ETL):** a Talend-style extract + load that lands four raw sources into the warehouse with audit columns and an ingestion-audit log.
- **Dimensional model (dbt):** a star schema — *fact_flights* and *fact_flight_delays* around conformed *dim_date*, *dim_airport* and *dim_airline* — with surrogate keys.
- **Data quality:** Great Expectations on the raw layer, 30 dbt tests on the modelled layers, and a source-to-target row-count reconciliation.
- **Orchestration:** an Airflow DAG for the daily batch plus a no-Docker Python runner for instant local demos.
- **Analytics:** four business marts (route performance, delay trends, cancellations, operational KPIs) surfaced in Power BI and Streamlit.

3. Architecture & data flow

Data moves through clearly separated layers, each with a single responsibility. Great Expectations guards the raw layer; dbt tests guard every modelled layer; reconciliation proves nothing is silently lost.

- **Sources** → **Ingestion:** extract profiles each file; load full-refreshes the raw schema idempotently and records rows-extracted vs rows-loaded.
- **Raw** → **Staging:** dbt views de-duplicate, null-out sentinel values, drop orphan-airport flights, standardize codes, and derive delay flags.
- **Staging** → **Dimensions/Facts** → **Marts:** the star schema is built and aggregated into analytics tables.
- **Marts** → **BI:** Power BI and Streamlit read the same marts, so both always agree.

4. The data model

A classic star schema at the grain of **one row per flight**. Facts hold measures and foreign keys; dimensions hold descriptive attributes. Surrogate keys are hashed so joins never depend on messy natural codes.

Table	Type	Grain / purpose
fact_flights	Fact	one row per flight; delay measures + FKs
fact_flight_delays	Fact	one row per flight × delay cause (unpivoted)
dim_date	Dimension	calendar attributes (conformed)
dim_airport	Dimension	airport + hub flag + region
dim_airline	Dimension	carrier + carrier type
route_performance / delay_trends	Mart	aggregated analytics
cancellations / operational_kpis	Mart	aggregated analytics

5. Data quality strategy

Quality is enforced in three complementary layers rather than assumed:

- **Great Expectations (raw):** nulls, uniqueness, schema/length, accepted values and ranges. Runs in monitor mode (reports defects) with a strict flag for a hard production gate.
- **dbt tests (modelled):** unique / not_null on keys, *relationships* tests that enforce fact→dimension integrity, accepted_values on categoricals, and custom singular tests.
- **Reconciliation:** accounts for every source row across source→raw→staging, classifying each delta as a removed duplicate or a DQ-filtered row; unexplained loss fails the build.

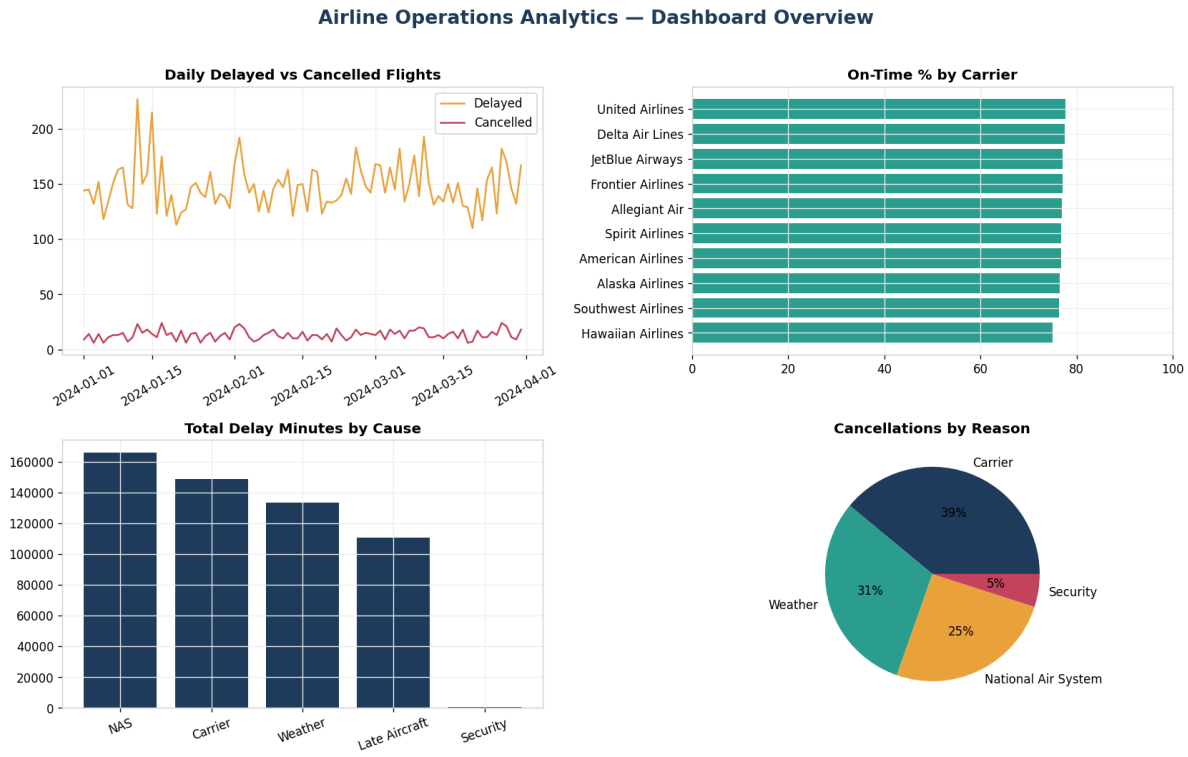
The synthetic data deliberately contains duplicates, -9999 sentinel delays, missing tail numbers and orphan airport codes, so these checks demonstrably catch and clean real defects.

6. Design decisions & trade-offs

Decision	Why
DuckDB default, Snowflake-ready	Zero-setup local runs; dbt makes the cloud swap a one-line profile change.
All logic in dbt	Warehouse-portable models; no warehouse-specific scripts to rewrite.
Full-refresh loads	Deterministic and simple at this volume; incremental is the noted next step.
GE in monitor mode	Shows cleaning of known-dirty raw data; --strict gives a production gate.
Synthetic seeded data	Self-contained + reproducible while exercising every DQ rule.

7. Results

From the default seeded run (~60k flights over one quarter): ~98% completion factor, ~77% on-time arrivals, ~7.5 min average arrival delay, with National Airspace and carrier issues the largest delay contributors and weather the dominant cancellation reason.



8. What this project teaches (skills to speak to)

- Designing a layered warehouse (raw → staging → marts) and why the split matters.
- Kimball dimensional modelling: facts, dimensions, grain, surrogate keys, conformed dimensions.
- Writing maintainable dbt: refs, sources, seeds, tests, macros with adapter dispatch.
- Operationalizing data quality with Great Expectations and reconciliation.
- Orchestrating a batch with Airflow and reasoning about task dependencies + retries.
- Making a project portable across warehouses and reproducible for anyone to run.

9. Interview talking points (Q & A)

Be ready to answer these confidently — they map directly to the repo.

Q. Walk me through the architecture.

A. Four sources are extracted and loaded into a raw schema with audit logging. Great Expectations validates the raw layer. dbt then transforms raw into staging (cleaning), a star schema (dims + facts), and analytics marts. A reconciliation step proves no rows are lost, and Airflow orchestrates the daily run. Power BI and Streamlit read the marts.

Q. What is the grain of your fact table?

A. One row per flight in fact_flights. fact_flight_delays is finer — one row per flight per contributing delay cause — because I unpivoted the five BTS delay-cause columns to make cause analysis a simple group-by.

Q. Why surrogate keys instead of the natural airport/carrier codes?

A. Surrogate keys (hashed) decouple the model from messy or changing natural keys, keep joins fast and uniform, and make it easy to introduce slowly-changing dimensions later without breaking fact joins.

Q. How do you guarantee data quality?

A. Three layers: Great Expectations on raw (nulls, uniqueness, schema, ranges), dbt tests on every model (including relationships tests that enforce referential integrity), and a source-to-target reconciliation that accounts for every row. Known-dirty inputs prove the checks actually fire.

Q. What is source-to-target reconciliation and why does it matter?

A. It compares row counts along source→raw→staging and explains every difference as either a removed duplicate or a DQ-filtered row. It catches silent data loss — the kind of bug that quietly corrupts a dashboard.

Q. How is this portable to Snowflake?

A. All transformation logic is in dbt, which abstracts the warehouse. Switching from DuckDB to Snowflake is a profile change plus credentials — the models, tests and business logic are identical. A dispatched macro handles the one dialect difference (title-casing).

Q. How would you make this incremental / handle years of data?

A. Make fact_flights an incremental model keyed on flight_date so only new partitions are processed, add dbt snapshots for slowly-changing dimensions, and stage raw files in S3 loaded to Snowflake. The full-refresh default was a deliberate simplicity choice for the demo.

Q. Why Airflow, and how are the tasks wired?

A. Airflow gives scheduling, retries and observability. The DAG runs generate → ingest → validate → dbt build → reconcile as a linear dependency chain; a failure in reconcile raises and fails the run so bad data never reaches BI.

Q. What would you improve next?

A. Incremental facts + SCD snapshots, a real BTS feed, a dbt docs/lineage site with freshness SLAs and alerting, and CI that runs dbt build + Great Expectations on every pull request.

10. How to run it

make setup → install · **make pipeline** → generate, ingest, validate, dbt build, reconcile · **make dashboard** → Streamlit · **make docker-up** → Airflow + dashboard. For Snowflake: set WAREHOUSE=snowflake and the SNOWFLAKE_* variables, then **dbt build --target snowflake**.

11. Putting it on your resume & LinkedIn

Resume bullet: Built an end-to-end airline analytics platform (Python, SQL, dbt, Airflow, Great Expectations, DuckDB/Snowflake, Power BI, Docker) with a tested star schema, three-layer data quality, and source-to-target reconciliation; local-first and cloud-portable.

LinkedIn line: Designed and shipped a production-shaped airline data pipeline — layered dbt modelling, enforced data quality, Airflow orchestration, and Power BI/Streamlit analytics — runnable on a laptop and portable to Snowflake.